

NLP Tutor

De Qwen2.5 à un assistant pédagogique spécialisé avec QLoRA

Comprendre → adapter → évaluer → démontrer

1. Problème, objectif et périmètre

Un assistant spécialisé doit être utile, mesurable... et savoir rester dans son domaine.

PROBLÈME

Un LLM généraliste sait parler de NLP, mais ses réponses peuvent être trop longues, irrégulières ou imprécises.

- Réponses en français
- Définition claire
- Explication progressive
- Exemple concret
- Phrase « À retenir »

OBJECTIF

Comparer deux systèmes

Baseline

Qwen2.5-1.5B-Instruct + prompting
→ aucun poids modifié

QLoRA

Même modèle + quantification 4 bits + adaptateurs LoRA entraînés

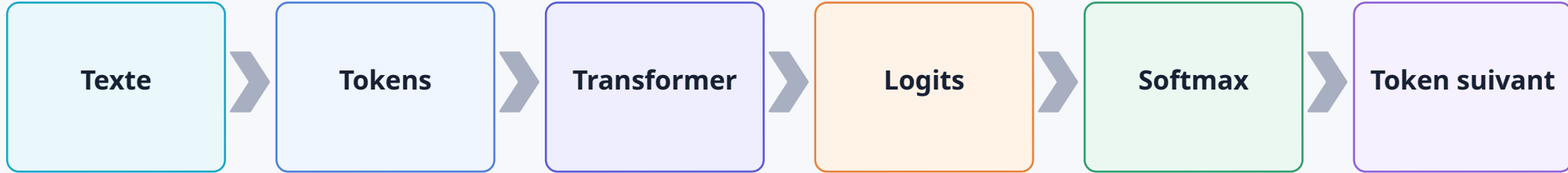
LIVRABLES

- Notebook exécuté
- Code modulaire dans src/nlp_tutor
- Dataset reproductible
- Adaptateur LoRA sauvegardé
- Résultats CSV avant/après
- Interface Gradio

Principe : système limité, cohérent, mesurable.

2. Ce qui se passe réellement quand le modèle répond

Avant l'adaptation, j'ai ouvert la "boîte noire" : tokens → logits → probabilités → décodage.



TOKENISATION

8 mots → 13 tokens

Un token peut être un mot, une partie de mot ou un signe. Le modèle manipule des identifiants numériques, pas directement des mots.

LOGITS + SOFTMAX

logits = scores bruts

softmax → probabilités dont la somme vaut 1. Le prochain token est choisi dans cette distribution.

DÉCODAGE

Greedy / température / top-k / top-p

Ces réglages ne changent pas ce que le modèle "sait", mais la manière dont il sélectionne les tokens et donc la diversité des réponses.

3. Baseline : que sait faire le modèle sans entraînement ?

Même modèle, mêmes fonctions de génération : seule l'adaptation changera ensuite.

SYSTEM PROMPT

Tu es NLP Tutor, un assistant pédagogique francophone spécialisé en NLP...

1. Réponds en français.
2. Commence par une définition claire.
3. Explique progressivement.
4. Donne un exemple.
5. Termine par « À retenir : ... ».

Aucun poids du modèle n'est modifié.

POURQUOI UNE BASELINE ?

- Mesurer le niveau initial du modèle
- Tester le prompt avant tout entraînement
- Conserver un point de comparaison identique
- Éviter d'attribuer à QLoRA ce que le modèle savait déjà

**Baseline = modèle pré-entraîné +
instructions + décodage**

4. Données : 500 couples question-réponse contrôlés

25 notions × 20 formulations ; séparation stricte train / validation / test.

350

TRAIN

14 formulations / concept

75

VALIDATION

3 formulations / concept

75

TEST

3 formulations / concept

25

CONCEPTS

token, softmax, QLoRA...

EXEMPLE DE DONNÉE

Concept : softmax

Question : « Explique-moi simplement softmax. »

Référence : « La softmax transforme un vecteur de scores en probabilités positives dont la somme vaut 1... »

Mots-clés : probabilités, somme, logits, distribution

LIMITE MÉTHODOLOGIQUE

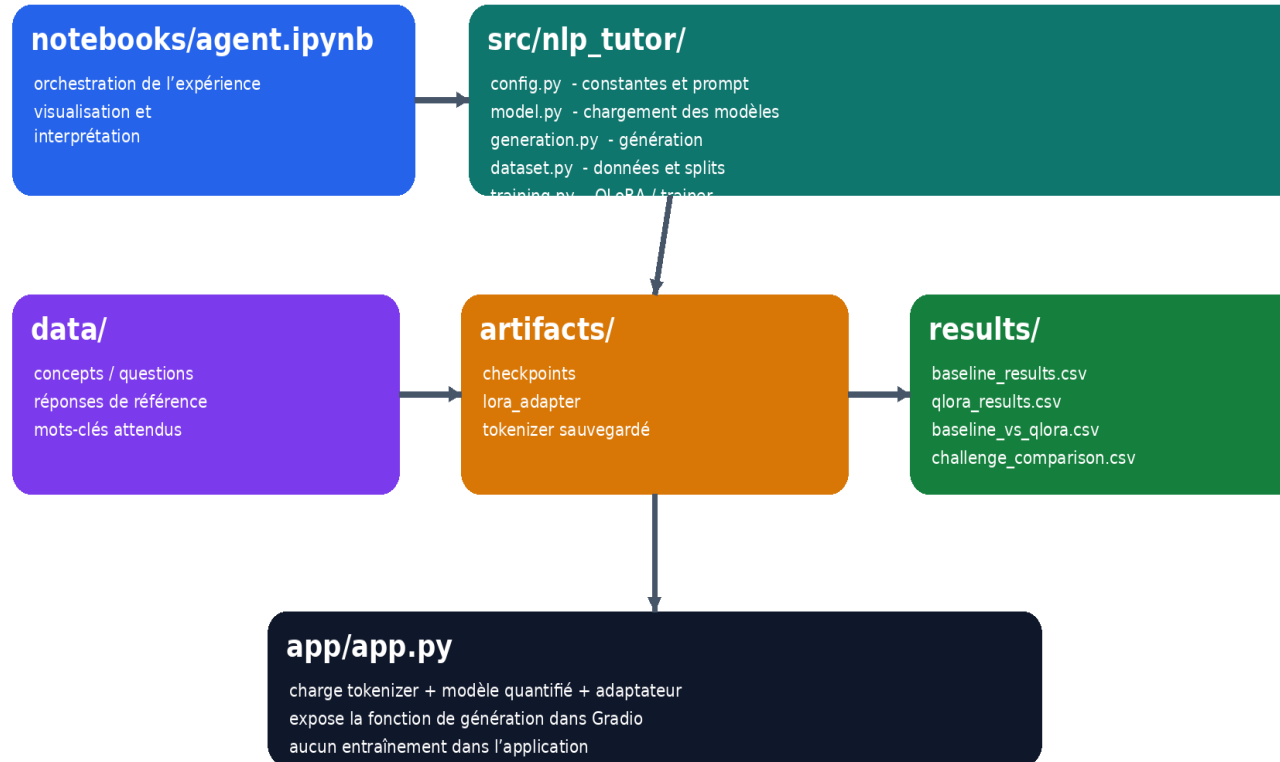
Les formulations changent, mais la réponse cible d'un même concept reste très similaire.

→ Le test standard mesure fortement l'adaptation au dataset ; il ne suffit pas à prouver la généralisation.

5. Organisation du code : chaque fichier a une responsabilité

Le notebook orchestre l'expérience ; la logique réutilisable est placée dans `src/nlp_tutor/`.

Architecture logicielle de NLP Tutor



config.py modèle, seed, prompt système

model.py tokenizer, base, QLoRA, adaptateur

generation.py chat template + stratégies de décodage

dataset.py concepts, templates, splits

training.py quantification, LoRA, SFTTrainer

evaluation.py ROUGE-L, coverage, CSV

app/app.py chargement final + Gradio

6. QLoRA : spécialiser le modèle sans tout réentraîner

QLoRA = modèle quantifié en 4 bits + adaptateurs LoRA entraînables.

DÉFINITION

Quantification

Les poids du modèle de base sont stockés en 4 bits pour réduire la mémoire.

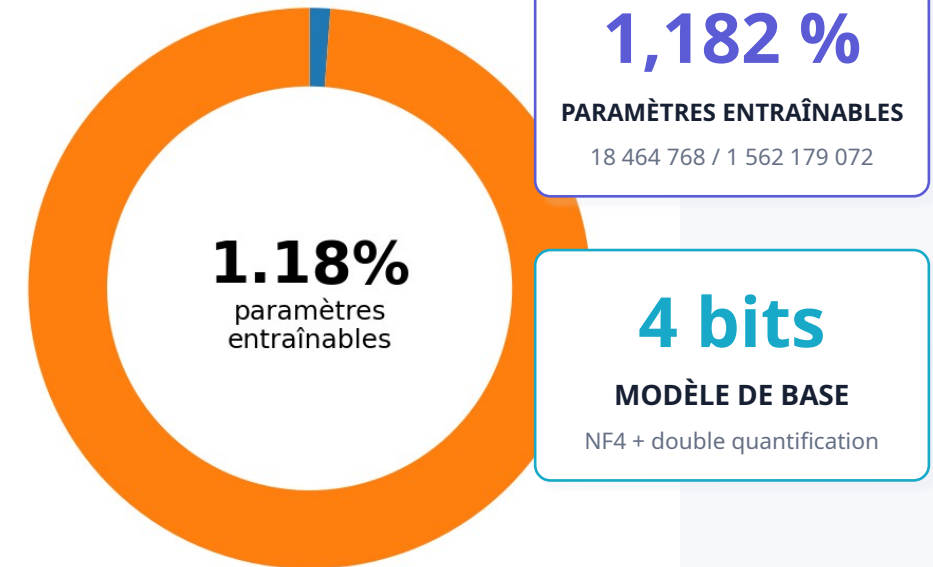
+

LoRA

On ajoute de petites matrices de faible rang. Les poids principaux restent gelés ; seuls les adaptateurs sont optimisés.

Pourquoi ? → Faire du fine-tuning sur une Tesla T4 sans entraîner 1,56 milliard de paramètres.

LoRA : petite fraction des poids entraînée



98,818 % des paramètres ne sont pas directement entraînés.

7. Fine-tuning : configuration réellement utilisée

Des réglages pensés pour la mémoire du T4 et un petit dataset pédagogique.

```
lora_config = LoraConfig(  
    r=16,  
    lora_alpha=32,  
    lora_dropout=0.05,  
    target_modules=[  
        "q_proj", "k_proj", "v_proj", "o_proj",  
        "gate_proj", "up_proj", "down_proj"  
    ]  
)
```

```
SFTConfig(  
    num_train_epochs=2,  
    per_device_train_batch_size=2,  
    gradient_accumulation_steps=8,  
    learning_rate=2e-4,  
    max_length=512,  
    gradient_checkpointing=True,  
    completion_only_loss=True  
)
```

2

epochs

16

batch effectif

2 × 8 accumulation

512

tokens max

2e-4

learning rate

8. Comment j'évalue : même test, mêmes métriques, avant et après

20 requêtes reproductibles issues du split test, puis un challenge séparé de généralisation.

PROTOCOLE

Même sélection de 20 requêtes

```
eval20 = dataset["test"]  
.shuffle(seed=123)  
.select(range(20))
```

- Baseline puis QLoRA
- Décodage déterministe pour comparer
- Résultats sauvegardés en CSV

ROUGE-L

Similarité textuelle

Mesure le recouvrement fondé sur la plus longue sous-séquence commune entre la prédiction et la référence.

**1,0 = très forte
correspondance textuelle
⇒ preuve de vérité ou de
généralisation**

COUVERTURE

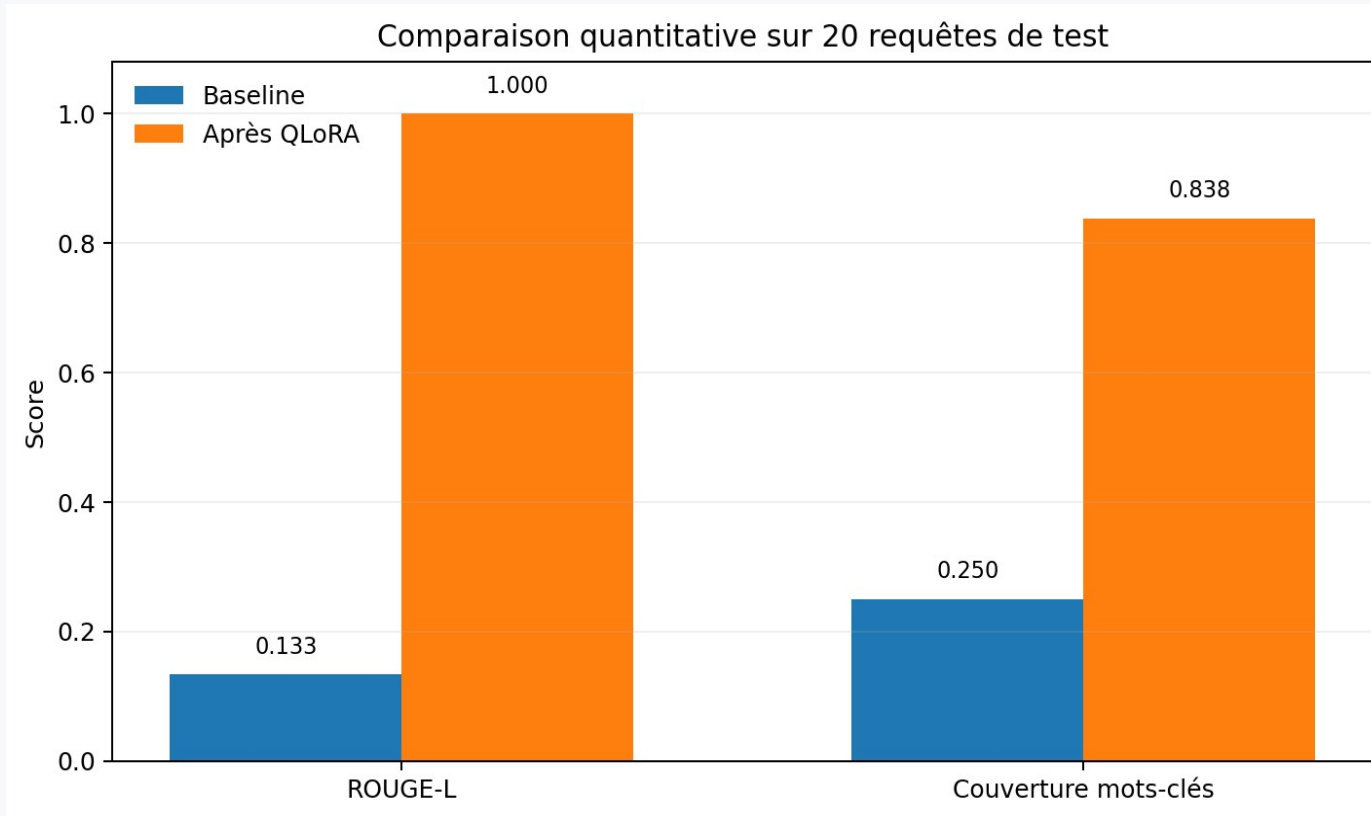
Mots-clés attendus

Pour chaque concept, une liste de mots-clés essentiels est définie. La métrique mesure la proportion présente dans la réponse.

```
coverage = mots_clés_présents  
/ mots_clés_attendus
```

9. Résultats : forte adaptation au format pédagogique

Les métriques automatiques progressent nettement après QLoRA.



0,1331

ROUGE-L · BASELINE

1,0000

ROUGE-L · QLoRA

$\Delta +0,8669$

0,2500

COVERAGE · BASELINE

0,8375

COVERAGE · QLoRA

$\Delta +0,5875$

Interprétation correcte

Le modèle s'est très fortement aligné sur les réponses de référence du dataset.

⚠ Pas « le modèle est parfait »

10. Test de généralisation : 20 questions formulées autrement

Objectif : sortir des templates du dataset et chercher les erreurs que ROUGE-L peut masquer.

QUESTION TYPE

« Quelle différence y a-t-il entre les logits et les probabilités finales ? »

Ce type de question demande une comparaison et pas seulement la récitation d'une définition.

Les sorties sont conservées dans :

```
results/challenge_comparison.csv
```

CE QUI S'AMÉLIORE

- Réponses souvent plus courtes
- Vocabulaire du cours mieux ciblé
- Format pédagogique plus stable
- Définitions fréquentes mieux apprises

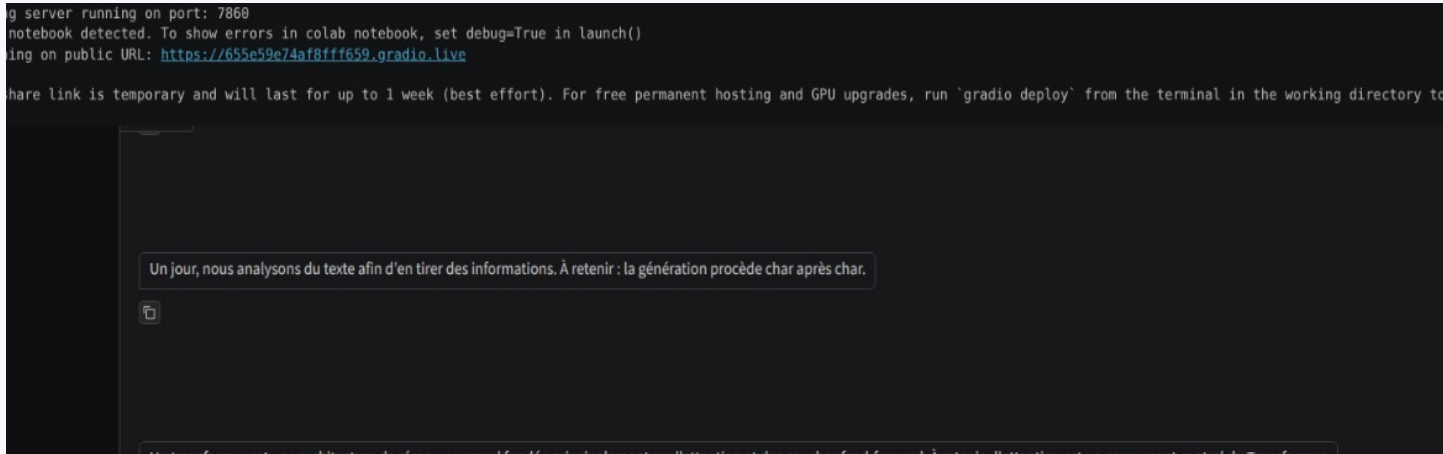
CE QUI RESTE DIFFICILE

- Questions de comparaison
- Raisonnement multi-étapes
- Certaines formulations très nouvelles
- Réponses parfois hors sujet malgré un bon score sur le test standard

Pas de score humain formel : inspection qualitative uniquement.

11. Démonstration : le modèle adapté dans Gradio

Déroulé : une définition connue, une question de compréhension, puis une question limite.



PLAN DE DÉMO

- 1 « Qu'est-ce qu'un token en NLP ? » vérifier le format pédagogique
- 2 « Explique-moi la self-attention simplement. » tester une notion centrale
- 3 « Quelle différence entre LoRA et QLoRA ? » tester une comparaison

CE QUE JE MONTRE

- Le modèle final est rechargé depuis l'adaptateur
- Pas de nouvel entraînement
- Réponse générée en direct
- Interface séparée du notebook
- Lien Gradio temporaire

Plan B en cas de réseau : utiliser la capture et les CSV sauvegardés.

12. Limites et améliorations prioritaires

Un bon projet ne s'arrête pas au meilleur score : il explique aussi ce qui reste fragile.

DONNÉES

- Réponses trop similaires
- Dataset synthétique et étroit
- Peu de questions multi-étapes

→ **Références plus diverses + vrais cas limites**

ÉVALUATION

- ROUGE-L ne mesure pas la vérité
- Pas de notation humaine formelle
- Challenge surtout qualitatif

→ **Grille humaine + test indépendant**

APPLICATION

- Historique Gradio non réinjecté
- Lien public temporaire
- Pas de gestion forte du hors-périmètre

→ **Multi-tour + garde-fous + déploiement**

Ce que le projet démontre

- 1 **Comprendre** tokenisation, logits, softmax, décodage
- 2 **Comparer** baseline vs modèle adapté
- 3 **Adapter** QLoRA : 1,182 % de paramètres entraînaables
- 4 **Évaluer** métriques automatiques + challenge
- 5 **Démontrer** application Gradio fonctionnelle

Résultat clé

0,1331 → 1,0000

ROUGE-L moyen

Mais : performance sur un test proche du dataset ≠ généralisation parfaite.

Questions ?

Merci.

ANNEXE A · Hyperparamètres exacts

Slide de secours pour les questions techniques.

Modèle	Qwen/Qwen2.5-1.5B-Instruct
Quantification	4 bits · NF4 · double quantification · compute float16
LoRA	r=16 · alpha=32 · dropout=0,05
Modules ciblés	q/k/v/o_proj + gate/up/down_proj
Epochs	2
Batch physique	2
Gradient accumulation	8 → batch effectif 16
Learning rate	2e-4
Warmup	10 steps
Max length	512 tokens
Gradient checkpointing	Oui
Loss	completion_only_loss=True
Seed	42

ANNEXE B · QLoRA vs prompting vs fine-tuning complet

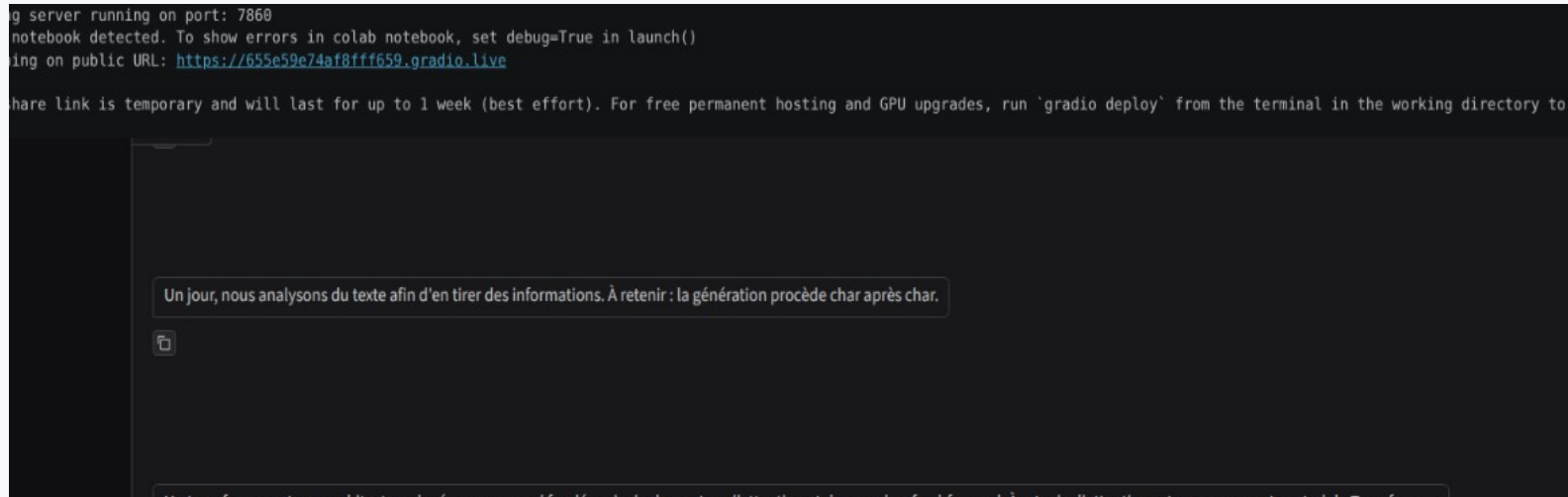
Réponse prête si le jury demande pourquoi ne pas avoir entraîné tout le modèle.

Approche	Poids modifiés	Mémoire	Avantage	Limite
Prompting	0	Faible	Très rapide à tester	Comportement parfois instable
LoRA / QLoRA	Petite fraction	Faible à modérée	Spécialisation efficace	Dépend fortement des données
Fine-tuning complet	Tous	Très élevée	Capacité d'adaptation maximale	Coûteux en GPU/mémoire

Choix du projet : QLoRA parce que le besoin est une spécialisation répétitive et les ressources sont limitées à un GPU T4.

ANNEXE C · Démo : plan de secours

À utiliser uniquement si le lien Gradio ou le runtime ne répond pas.



SI LA DÉMO PLANTE

- Montrer cette capture
- Ouvrir les CSV sauvegardés
- Expliquer que le lien Gradio est temporaire
- Ne pas relancer l'entraînement

baseline_results.csv
qlora_results.csv
baseline_vs_qlora.csv
challenge_comparison.csv

Les résultats et l'adaptateur ont été sauvegardés précisément pour rendre la présentation indépendante d'un nouveau run d'entraînement.

ANNEXE D • Définitions Transformer à connaître

Rappels de cours utiles pour répondre précisément au jury.

Embedding

Vecteur numérique associé à un token. Il transforme un identifiant discret en représentation exploitable par le réseau.

Self-attention

Mécanisme qui permet à chaque position de pondérer l'information provenant des autres positions autorisées de la même séquence.

Query / Key / Value

Q exprime ce qu'une position recherche ; K sert à calculer la compatibilité ; V porte l'information effectivement agrégée.

Masque causal

Empêche une position de consulter les tokens futurs. La prédiction du prochain token ne dépend que du contexte précédent.

Pré-entraînement

Apprentissage général sur un grand corpus avant la spécialisation à une tâche ou à un format particulier.

Instruction tuning

Fine-tuning sur des couples instruction-réponse afin d'améliorer le suivi des consignes et le format conversationnel.

Transformer causal = embeddings + information de position + blocs d'attention/MLP + masque causal + tête de prédiction du prochain token.

ANNEXE E • Questions probables du jury

Réponses courtes à garder en réserve pendant les échanges.

Pourquoi QLoRA ?

Pour réduire la mémoire : modèle de base quantifié + seulement une petite fraction de paramètres entraînaibles.

Pourquoi seulement 2 epochs ?

Dataset petit et répétitif ; davantage d'epochs augmente le risque de surapprentissage sans garantie de meilleure généralisation.

Pourquoi ROUGE-L = 1 ? Le modèle reproduit très bien des références très similaires entre reformulations. Ce score mesure la proximité textuelle, pas la vérité.

Pourquoi 20 requêtes ?

La consigne demande au moins 20 requêtes ; la sélection est reproductible et identique pour les deux configurations.

Que manque-t-il à l'application ?

Le véritable multi-tour : Gradio fournit history, mais il n'est pas encore réinjecté dans le contexte du modèle.

Comment améliorer l'évaluation ?

Créer un test indépendant plus divers, ajouter une grille humaine et mesurer aussi les demandes hors périmètre.